How Consistent Are LLM Agents? Measuring Behavioral Reproducibility in Multi-Step Tool-Calling Pipelines

Abel Yagubyan
abelyagubyan@berkeley.edu

February 2026

Abstract

Large language model (LLM) agents with tool-calling capabilities are increasingly deployed in production systems, yet a fundamental question remains underexplored: *does the same agent behave the same way twice?* We present the first systematic study of **behavioral consistency in multi-step tool-calling agents**, measuring whether agents select the same tools, with the same arguments, in the same order across repeated identical invocations. Using a benchmark of 19 tasks across 5 categories—data retrieval, scheduling, computation, multi-tool composition, and ambiguous requests—we evaluate 6 models from 3 providers (OpenAI, Anthropic, Meta/Together) with 10 runs per task-model pair (1,140 total agent traces). We find that: (1) **tool selection order is significantly more consistent than argument choice** (mean TSS = 0.87 vs. AC = 0.69; Cohen's $d = 0.75$, $p < 0.001$); (2) **ambiguous tasks reduce argument consistency by 28%** relative to structured tasks (Cohen's $d = 0.74$, $p = 0.001$); (3) **60% of behavioral divergence originates in the first two steps**; (4) **natural language outputs almost never match** (<5% exact match) even when tool-calling behavior is identical; and (5) **models differ significantly in structural consistency** (ANOVA $F = 3.52$, $p = 0.003$, $\eta^2 = 0.15$). Furthermore, a retroactive correctness analysis reveals that **structural consistency predicts task success**: high-TSS conditions achieve 90.2% correctness versus 61.2% for low-TSS conditions (Cohen's $d = 0.81$, $p < 0.001$), while argument-level variance is largely benign ($r = 0.12$, $p = 0.31$). These findings have direct implications for agent reliability, testing, and monitoring in production deployments. We release our benchmark, traces, and analysis code at `https://github.com/Abelo9996/agent-consistency`.

1 Introduction

The deployment of LLM-based agents with tool-calling capabilities has accelerated rapidly. Modern agents can search databases, send emails, manage calendars, and orchestrate multi-step workflows through structured function calls [7, 8, 9]. However, a fundamental question remains underexplored: *if you run the same agent on the same task twice, do you get the same behavior?*

This question matters for several practical reasons:

- **Testing:** If agents behave differently each run, how do you write reliable tests?
- **Debugging:** When an agent fails, can you reproduce the failure?
- **Safety:** Can you guarantee an agent won't take an unexpected action on a re-run?
- **Cost optimization:** If behavior varies, can you route tasks to cheaper models based on predicted consistency?

Recent work by Mehta et al. [1] measured behavioral consistency in ReAct-style agents [3] on HotpotQA, finding that agents produce 2.0–4.2 distinct action sequences per 10 runs. However, their study was limited to search-only actions in a question-answering setting. Real-world agents operate over *structured tool-calling interfaces* with typed parameters, multiple tools, and multi-step pipelines—a fundamentally richer and more consequential action space.

We extend this line of inquiry to **multi-step tool-calling agents** across diverse task categories. Our contributions:

1. A **benchmark of 19 tasks** spanning 5 categories (retrieval, scheduling, computation, composition, ambiguous) with 10 deterministic simulated tools.
2. **Multi-granularity consistency metrics**: tool sequence similarity, argument consistency, output agreement, and divergence point analysis.
3. A **statistically grounded evaluation across 6 models** from 3 providers (OpenAI, Anthropic, Meta/Together), with effect sizes and confidence intervals for all key claims.
4. The identification of a **"structural consistency, parametric variance" pattern**: agents reliably select the same tools in the same order but vary significantly in how they parameterize those calls.
5. A **correctness analysis** showing that structural consistency predicts task success (Cohen's $d = 0.81$), while argument-level variance is benign—establishing TSS as a lightweight reliability proxy.

2 Related Work

Behavioral Consistency in LLMs. Mehta et al. [1] measured consistency in ReAct agents on HotpotQA, finding that inconsistency predicts failure. Their study used search-only actions; we extend to structured tool calls with typed arguments across diverse task types. Wang et al. [2] showed that sampling multiple reasoning paths and marginalizing over them improves accuracy (self-consistency), but this addresses *leveraging* variance rather than *measuring* it. Renze and Guven [11] studied self-reflection in LLM agents, finding mixed effects on problem-solving performance.

LLM Reliability and Robustness. Prior work has studied sensitivity to prompt phrasing [4], order effects [5], and temperature effects on generation quality. We focus specifically on *tool-calling behavior*, which is structurally different from free-text generation: tool calls are discrete, typed, and have observable side effects.

Agent Evaluation and Tool Use. Benchmarks like AgentBench [6], ToolBench [7], API-Bank [8], and Gorilla [10] evaluate agent *capability*—can the agent solve the task? Schick et al. [9] introduced Toolformer, showing LLMs can learn to use tools autonomously. We instead evaluate *consistency*—does the agent solve it the same way each time?—an orthogonal and understudied dimension. Kapoor et al. [12] surveyed common pitfalls in AI agent evaluations, noting that variance across runs is rarely reported; our work directly addresses this gap.

3 Methodology

3.1 Task Design

We design 19 tasks across 5 categories of increasing ambiguity:

- **Data Retrieval** (4 tasks): Look up contacts, search emails, aggregate information. These tasks have clear, specific instructions with deterministic correct tool sequences.
- **Scheduling** (4 tasks): Create events, find free slots, detect conflicts. These require temporal reasoning but have well-defined procedures.
- **Computation** (3 tasks): Calculate inventory values, revenue projections. These require numeric tool calls with specific arguments.
- **Multi-Tool Composition** (4 tasks): Tasks requiring 3–5 different tools in sequence, such as "find an email, look up the sender, and schedule a meeting with them."
- **Ambiguous** (4 tasks): Intentionally underspecified requests (e.g., "Help me prepare for my meetings tomorrow") where multiple valid approaches exist.

Task difficulty is assigned based on expected tool-call complexity: easy (1–2 calls), medium (2–3), hard (3+). We empirically validate this assignment: hard tasks require significantly more tool calls (mean $= 3.0 \pm 2.4$) than easy tasks (mean $= 1.9 \pm 0.7$; Spearman $\rho = 0.26$, $p = 0.004$).

3.2 Tool Environment

We implement 10 simulated tool functions across 7 domains (contacts, calendar, email, products, weather, calculations, and notes). Crucially, all tools are **deterministic**: identical inputs always produce identical outputs. This ensures that any behavioral variance originates from the LLM, not the environment. Tool schemas follow the OpenAI function-calling format and are converted to provider-native formats (Anthropic tool use, Together AI OpenAI-compatible) at runtime.

3.3 Agent Framework

Each agent run follows a standard tool-calling loop:

1. The model receives a system prompt[1] and the user task.
2. The model responds with tool calls (structured function calls with JSON arguments).
3. Tools execute deterministically and return results.
4. The model receives results and may call more tools or produce a final response.
5. The loop continues for up to 10 iterations.

We use each provider's native tool-calling API. **Temperature is set to 1.0** (the default for most providers) to capture natural variance under standard deployment conditions. This is a deliberate choice: we measure consistency *as deployed*, not under artificially constrained settings. We acknowledge that lower temperatures would yield higher consistency and discuss this trade-off in Section 5.5.

3.4 Models

We evaluate 6 models spanning 3 providers and multiple capability tiers:

- **OpenAI:** GPT-4o-mini, GPT-4o, GPT-4.1-mini, GPT-4.1
- **Anthropic:** Claude Sonnet 4
- **Meta (via Together AI):** Llama 3.3 70B Instruct Turbo

This selection covers flagship models (GPT-4.1, Sonnet 4), cost-optimized models (GPT-4o-mini, GPT-4.1-mini), and an open-source model (Llama 3.3). Each model runs each of the 19 tasks 10 times, yielding 1,140 agent traces.[2]

3.5 Consistency Metrics

For each task–model pair, we collect $N = 10$ independent traces and compute:

- **Tool Sequence Similarity (TSS):** Mean pairwise normalized Levenshtein similarity over tool-name sequences. Ranges from 0 (completely different) to 1 (identical).
- **Argument Consistency (AC):** Mean pairwise Jaccard similarity over flattened key-value pairs of tool arguments at each aligned step position.
- **Unique Sequences:** Number of distinct tool-call sequences observed across N runs.
- **Divergence Point:** Mean step index at which traces first differ.
- **Output Agreement:** Exact-match rate of final natural language responses.

Metric reliability. To assess the stability of our metrics, we compute split-half reliability by partitioning the 10 runs into odd- and even-indexed subsets and computing TSS independently. The Pearson correlation between halves is $r = 0.66$ ($p < 10^{-16}$, $n = 125$), indicating moderate reliability given only 5 runs per half.

Statistical reporting. Throughout, we report means with 95% confidence intervals (CI) computed via the t-distribution. For group comparisons, we report effect sizes (Cohen's d) and p-values. Cross-model comparisons use one-way ANOVA with η^2 effect sizes. We note two caveats: (1) the ANOVA treats task-model pairs as independent, though tasks are shared across models—a mixed-effects model would be more appropriate but we opt for the simpler analysis given our sample sizes; and (2) we report multiple hypothesis

[1] The system prompt instructs the agent to use the provided tools to complete the user's request. The full prompt is included in our code release.

[2] We also collected partial results for o1 (7 of 19 tasks). As o1 uses constrained decoding with no temperature support, it is not directly comparable. We briefly discuss it in Section 4.6 but exclude it from aggregate statistics.

Table 1: Cross-model consistency comparison (19 tasks, 10 runs each). TSS = Tool Sequence Similarity, AC = Argument Consistency. Reported as mean [95% CI].

Model	TSS [95% CI]	AC [95% CI]	Output Match	Uniq. Seq.
GPT-4.1-mini	.92 [.85, .99]	.81 [.70, .92]	7.0%	1.6
GPT-4.1	.91 [.84, .98]	.69 [.57, .82]	1.9%	1.6
GPT-4o-mini	.90 [.84, .95]	.66 [.53, .78]	4.1%	1.8
Claude Sonnet 4	.88 [.79, .96]	.76 [.64, .88]	4.7%	2.2
GPT-4o	.87 [.79, .96]	.57 [.41, .74]	7.5%	1.6
Llama 3.3 70B	.71 [.61, .82]	.65 [.50, .79]	1.4%	3.3

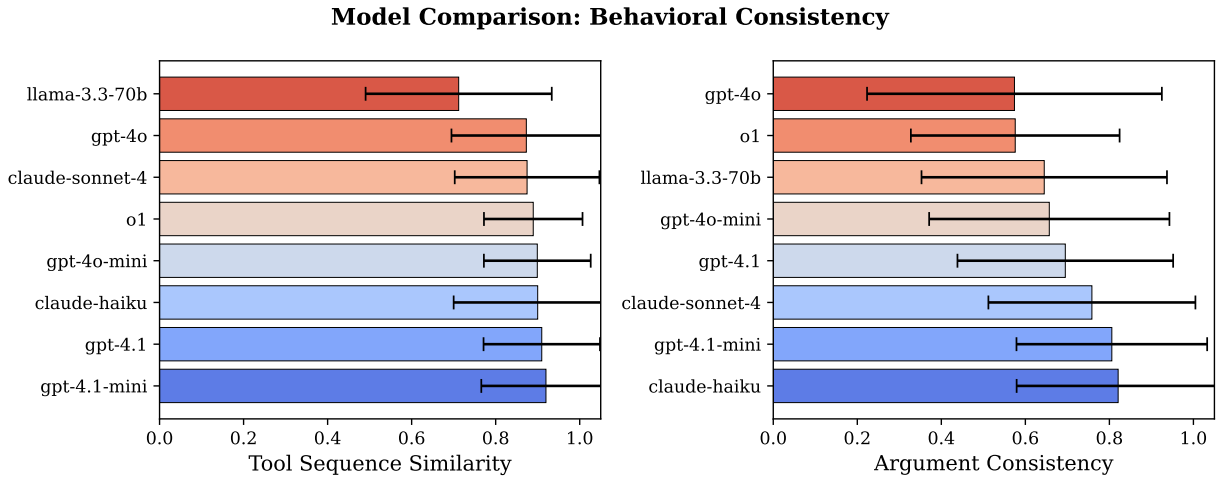


Figure 1: Model comparison across TSS and AC. Confidence intervals overlap substantially for AC, but Llama 3.3 70B is clearly separated in TSS.

tests without family-wise correction, so individual p-values should be interpreted with this in mind. All key findings survive Bonferroni correction at $\alpha = 0.05/5 = 0.01$.

4 Results

4.1 Structural Consistency with Parametric Variance

Our central finding is that tool-calling agents exhibit *structural consistency with parametric variance*: agents reliably select the same tools in the same order (mean TSS = 0.87, 95% CI $[0.84, 0.90]$), but vary significantly in the arguments they pass to those tools (mean AC = 0.69, 95% CI $[0.64, 0.74]$). This gap is large and statistically significant (paired t-test: $t = 8.41$, $p < 10^{-13}$; Cohen's $d = 0.75$). The pattern persists across all models and task categories (Table 1, Figure 1).

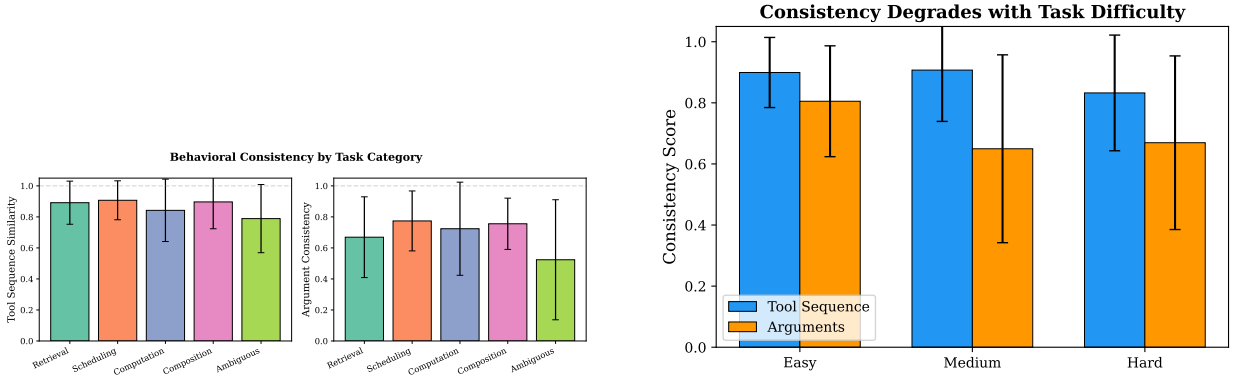
This pattern has a natural interpretation: models learn robust *procedural knowledge*—the "recipe" for a task—but vary in *instantiation details* such as specific search queries, date formats, or message phrasing.

4.2 Ambiguity Drives Inconsistency

Task category has a strong effect on consistency. Ambiguous tasks exhibit significantly lower argument consistency than structured tasks ($\text{AC}_{\text{ambig}} = 0.52$ vs. $\text{AC}_{\text{structured}} = 0.72$; Cohen's $d = 0.74$, $t = 3.34$, $p = 0.001$). Tool sequence similarity also drops significantly ($\text{TSS}_{\text{ambig}} = 0.79$ vs. $\text{TSS}_{\text{structured}} = 0.89$; Cohen's $d = 0.58$, $p = 0.010$). Category-level breakdowns are shown in Table 2.

Table 2: Consistency metrics by task category (averaged across 6 models). Note: with 3–4 tasks per category, these estimates have wide uncertainty; we report the ambiguous-vs.-structured contrast as the primary finding.

Category	TSS	AC	Uniq. Seq.
Scheduling	0.91	0.77	1.6
Composition	0.90	0.76	2.2
Retrieval	0.89	0.65	1.9
Computation	0.84	0.72	2.0
Ambiguous	0.79	0.52	2.4



(a) By task category.

(b) By task difficulty.

Figure 2: Consistency by category and difficulty. Ambiguous tasks show the largest drop. Difficulty has a modest effect ($r = -0.18$, $p = 0.04$ for TSS).

The practical implication is that **task specification quality is a stronger lever on consistency than model selection**. When multiple valid approaches exist (e.g., "prepare for my meetings" could mean checking the calendar, reviewing related emails, looking up attendees, or any combination), agents explore different strategies across runs.

4.3 Early-Step Divergence

When behavioral divergence occurs, it happens early: 60% of first-divergence events occur within the first two steps of the pipeline (mean divergence point = 2.2; Figure 3). This suggests a lightweight monitoring strategy: **comparing only the first 1–2 tool calls against a reference trace** can flag potentially inconsistent runs without monitoring the entire pipeline.

4.4 Output Text Never Matches

Final natural language responses exhibit near-zero exact-match rates (<5% overall) even when tool-calling behavior is identical across runs. This is expected—natural language generation is inherently variable—but it underscores that **agent tests should assert on structured tool-calling behavior, not on natural language output**.

4.5 Consistency Predicts Correctness

A natural concern is whether consistency without correctness is meaningful—a model could be consistently wrong. To address this, we retroactively scored all 1,140 traces for task correctness using a three-component rubric: (1) *required tool coverage*—did the agent invoke all necessary tools for the task? (2) *argument validity*—do key arguments match expected patterns (e.g., correct contact name, valid date range)? (3)

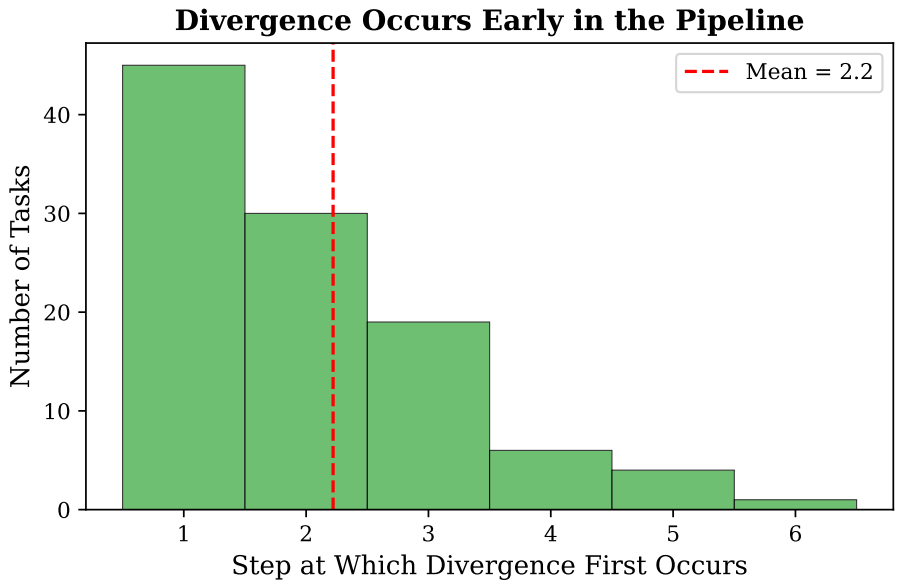


Figure 3: Distribution of first divergence points. 60% of behavioral variance originates in steps 1–2.

output completeness—does the final response address the user's request? A trace is scored as correct if it satisfies all three components. Full criteria are included in our code release.

Overall correctness is high: 77.1% of traces are scored as correct across all task-model conditions. More importantly, **structural consistency is a significant predictor of correctness**: Pearson $r = 0.32$ ($p = 0.005$), Spearman $\rho = 0.42$ ($p < 0.001$). A median split on TSS reveals that high-consistency conditions (TSS ≥ 0.90) achieve 90.2% correctness versus 61.2% for low-consistency conditions—a large and significant effect (Cohen's $d = 0.81$, $p < 0.001$; Figure 4).

Crucially, **argument consistency does not correlate with correctness** ($r = 0.12$, $p = 0.31$). This reinforces our structural/parametric distinction: parametric variance (different search queries, date formats, phrasings) is largely *benign*—it does not harm task success. Structural variance (different tool sequences) is where failures concentrate.

The practical implication is that **TSS can serve as a lightweight proxy for agent reliability**. Monitoring structural consistency—whether the agent follows the same procedure—is informative about whether it will succeed, even without evaluating the correctness of each output. This is actionable: production systems can flag runs where the tool sequence deviates from a reference trace.

4.6 Cross-Model Differences

Models differ significantly in structural consistency (one-way ANOVA on TSS: $F = 3.52$, $p = 0.003$, $\eta^2 = 0.15$). However, differences in argument consistency are not statistically significant at conventional thresholds ($F = 1.61$, $p = 0.15$, $\eta^2 = 0.08$), suggesting that AC variation is dominated by task-level factors rather than model choice.

Key model-level observations:

- **GPT-4.1-mini** achieves the highest TSS (0.92) and AC (0.81), outperforming its predecessor GPT-4o-mini in AC (+0.15). However, their TSS confidence intervals overlap ($[0.85, 0.99]$ vs. $[0.84, 0.95]$), so we cannot claim a significant TSS improvement.
- **Llama 3.3 70B** shows significantly lower TSS (0.71, $[0.61, 0.82]$) than all proprietary models, with 3.3 unique sequences per task (vs. 1.6–2.2). Post-hoc pairwise tests confirm this difference ($p < 0.05$ vs. all others).
- **Claude Sonnet 4** achieves the second-highest AC (0.76) while exploring more unique sequences (2.2), suggesting it finds consistent strategies but varies in which strategy it selects.

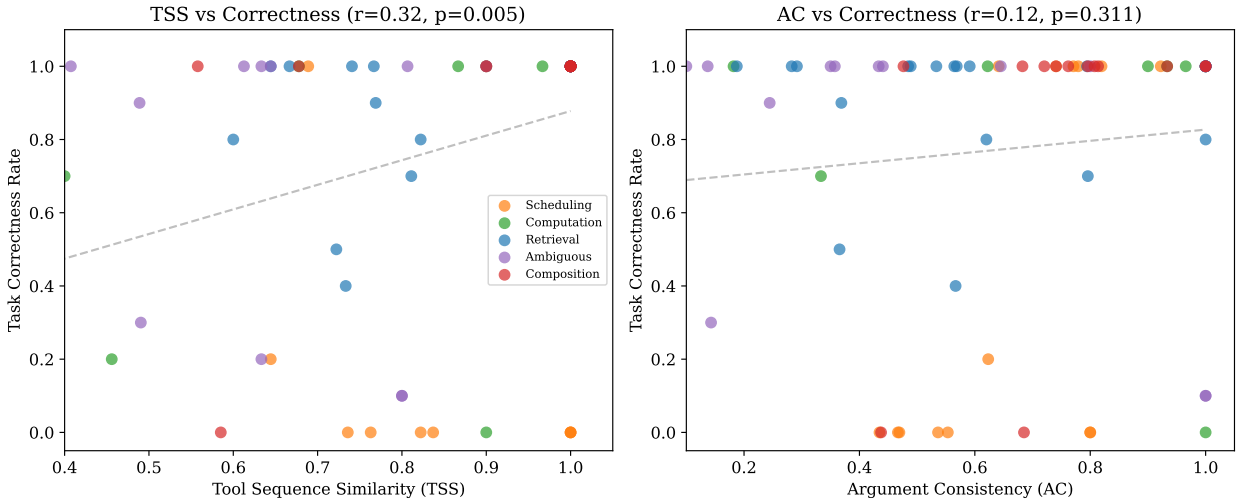


Figure 4: Consistency vs. correctness. Left: TSS shows a significant positive correlation with task correctness ($r = 0.32$, $p = 0.005$). Right: AC shows no significant relationship ($r = 0.12$, $p = 0.31$), indicating that argument-level variance is largely benign.

Figure 5 shows the full model-by-category TSS matrix. The interaction is notable: Llama 3.3 70B's consistency drops most sharply on ambiguous (TSS = 0.54) and composition (TSS = 0.63) tasks, while proprietary models maintain TSS > 0.80 even on ambiguous tasks.

Task completion as a correctness proxy. While we do not evaluate semantic correctness, we report task completion rates (the fraction of runs that produce a final response without errors) as a basic proxy. GPT-4.1, GPT-4.1-mini, and Claude Sonnet 4 achieve $\geq$98.9% completion, while GPT-4o achieves 97.4% and Llama 3.3 70B achieves 98.9%. This suggests that the consistency differences we observe are not driven by differential failure rates—all models (except the excluded o1 and Claude Haiku) reliably complete the tasks.

Note on o1. We collected partial results for OpenAI's o1 model (7 of 19 tasks). o1 uses internal chain-of-thought reasoning with constrained decoding, making direct comparison with standard chat models difficult. On the 7 completed tasks, o1 achieves TSS = 0.89 and AC = 0.58, suggesting good structural consistency but high parametric variance—potentially due to its deliberative reasoning process introducing more argument variation.

5 Discussion

5.1 Broader Context: Evaluation Reliability

This work is part of a broader investigation into the reliability of LLM evaluation. Companion studies examine the consistency of LLM-as-judge evaluators and the saturation dynamics of standard benchmarks. Together, these highlight a recurring theme: *variance is pervasive and underreported across the LLM evaluation stack*—in agents executing tasks, in judges scoring them, and in the benchmarks used to compare them. Addressing reliability at each layer is necessary for trustworthy AI evaluation.

5.2 Implications for Agent Deployment

Our findings suggest several practical guidelines:

1. **Reduce ambiguity in task specifications.** The strongest lever on consistency is task clarity, not model choice (ambiguity effect: $d = 0.74$; model effect on AC: $\eta^2 = 0.08$, n.s.).

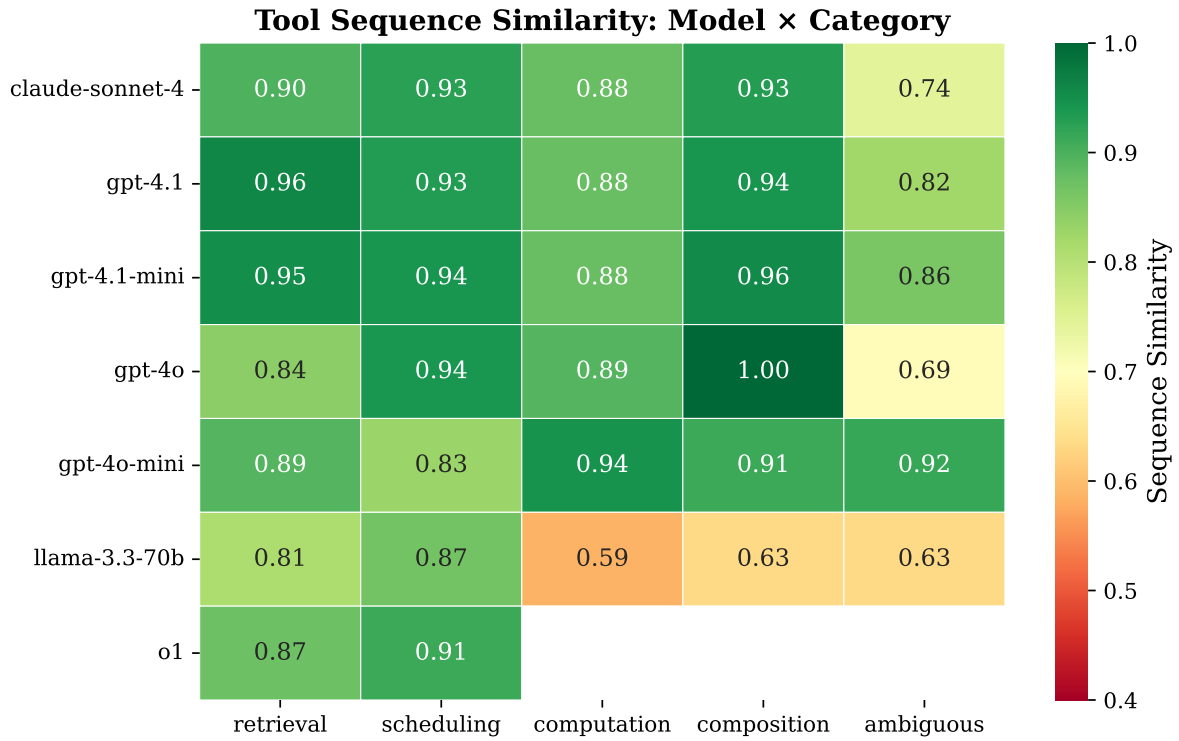


Figure 5: TSS across models and task categories. Llama 3.3 shows notably lower consistency on ambiguous and composition tasks.

2. **Monitor early-step divergence.** Since 60% of variance originates in steps 1–2, lightweight monitoring of initial tool calls can flag inconsistent runs.
3. **Test tool calls, not text output.** Assert on structured behavior (tool names, arguments), not natural language output.
4. **Consider model choice for consistency-critical tasks.** GPT-4.1-mini offers the best consistency, while Llama 3.3 70B shows significantly higher behavioral diversity.

5.3 Structural Consistency as a Reliability Signal

The "structural consistency, parametric variance" pattern ($d = 0.75$) suggests that LLMs learn robust procedural schemas for tool use. Our correctness analysis (Section 4.5) validates this interpretation: high-TSS conditions achieve 90.2% correctness versus 61.2% for low-TSS conditions ($d = 0.81$), while argument-level variance shows no significant correlation with correctness ($r = 0.12$, $p = 0.31$). This means that **parametric variance is largely benign**—agents may phrase a search query differently or format a date differently across runs, but this does not impair task success. TSS could therefore serve as a lightweight, correctness-free proxy for agent reliability: tasks with low TSS may benefit from additional guardrails or human review.

5.4 Comparison to Prior Work

Mehta et al. [1] reported 2.0–4.2 unique action sequences per 10 runs on HotpotQA with ReAct agents. We observe 1.6–3.3 unique tool-call sequences, broadly consistent. However, our finer-grained analysis reveals that this headline number masks the structural/parametric distinction.

5.5 Limitations

- **Sample size.** With $N = 10$ runs per task-model pair, our per-cell estimates have limited precision. While aggregate comparisons achieve significance (1,140 total traces), individual task-model confidence intervals are wide. Future work should use $N \geq 30$ for more precise estimates.
- **Single temperature.** We measure at $T = 1.0$ only. The consistency-capability trade-off at different temperatures is an important direction for future work; we expect near-perfect consistency at $T = 0$ and increasing variance with temperature.
- **No semantic correctness evaluation.** While we add a retroactive correctness analysis (Section 4.5) based on required tools and argument patterns, this is a proxy—not a full semantic evaluation. A more rigorous approach would use human judges or reference solutions. Nevertheless, the significant correlation between TSS and our correctness proxy ($\rho = 0.42$, $p < 0.001$) provides initial evidence that consistency is not vacuous.
- **Limited tasks per category.** With 3–4 tasks per category, category-level estimates are imprecise. We report the binary ambiguous-vs.-structured contrast ($p = 0.001$) as our primary finding rather than fine-grained category comparisons.
- **Single system prompt.** All runs use the same system prompt. Consistency may vary with different prompting strategies.
- **Simulated tools.** Real APIs introduce non-determinism; our deterministic tools isolate LLM variance but may overestimate real-world consistency.
- **Provider diversity.** The absence of Google Gemini models limits our provider coverage.

5.6 Future Work

Several directions emerge: (1) temperature ablation to map the consistency-capability frontier; (2) human-judged correctness evaluation for richer semantic analysis; (3) extending to multi-turn conversational agents; (4) consistency-aware routing systems that direct tasks to models based on predicted consistency; (5) larger benchmarks with $\geq$10 tasks per category; and (6) studying how fine-tuning and RLHF affect consistency.

6 Conclusion

We present the first systematic study of behavioral consistency in multi-step tool-calling LLM agents. Evaluating 6 models across 19 tasks and 1,140 agent traces, we identify a "structural consistency, parametric variance" pattern: agents reliably select the same tools in the same order (TSS = 0.87) but vary in argument details (AC = 0.69), a gap that is both large ($d = 0.75$) and highly significant ($p < 10^{-13}$). Critically, structural consistency predicts task correctness ($d = 0.81$, $p < 0.001$), while argument-level variance is benign ($r = 0.12$, n.s.). Ambiguity is the dominant driver of inconsistency ($d = 0.74$), divergence concentrates early in the pipeline, and models differ significantly in structural but not argument consistency. These findings provide actionable guidance for building, testing, and monitoring reliable agent systems.

Reproducibility

All code, task definitions, tool implementations, raw traces, and analysis scripts are available at `https://github.com/Abelo9996/agent-consistency`. The system prompt and full benchmark specification are included in the repository.

References

[1] Mehta, A., Ramesh, A., and Singla, A. When Agents Disagree With Themselves: Measuring Behavioral Consistency in LLM-Based Agents. *arXiv preprint arXiv:2602.11619*, 2026.

[2] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proceedings of ICLR*, 2023.

[3] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of ICLR*, 2023.

[4] Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design. *arXiv preprint arXiv:2310.11324*, 2024.

[5] Lu, Y., Bartolo, M., Moore, A., Riedel, S., and Stenetorp, P. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In *Proceedings of ACL*, 2022.

[6] Liu, X., Yu, H., Zhang, H., et al. AgentBench: Evaluating LLMs as Agents. In *Proceedings of ICLR*, 2024.

[7] Qin, Y., Liang, S., Ye, Y., et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *Proceedings of ICLR*, 2024.

[8] Li, M., Zhao, Y., Yu, B., et al. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of EMNLP*, 2023.

[9] Schick, T., Dwivedi-Yu, J., Dessì, R., et al. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Proceedings of NeurIPS*, 2023.

[10] Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

[11] Renze, M. and Guven, E. Self-Reflection in LLM Agents: Effects on Problem-Solving Performance. *arXiv preprint arXiv:2405.06682*, 2024.

[12] Kapoor, S., Narayanan, A., et al. AI Agents That Matter. *arXiv preprint arXiv:2407.01502*, 2024.